

Week 5 - Lecture 10 - Problem: Cannon and Projectile Types

ILO

- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, classes, and methods.
- **CS2:** Structure programs into manageable and reusable units.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS5:** Express mathematical formulas as programming expressions and algorithms.

Objectives

After completing this problem, you should be able to:

1. Move behavior to the class that is responsible for it.
2. Model an aggregation relationship between a **Cannon** and a **Projectile**.
3. Create child classes using inheritance and `super()`.
4. Override a method to produce polymorphic behavior.
5. Inspect the instance attributes of an object with `__dict__`.

Background

In Lecture 9, a **Projectile** object both stored its position and performed the firing simulation. This works, but firing is more naturally the responsibility of a **Cannon**: a cannon launches an object and updates its position during flight.

A cannon can be given a projectile that was created elsewhere. If the cannon stores that object, the relationship is **aggregation**: the cannon has a projectile, but the projectile can still exist without that cannon.

Different projectiles can share common data, such as an identifier, size, energy, and position. A **Bomb** and a **Missile** can therefore inherit from a shared **Projectile** parent class. Each child adds its own data and can provide its own version of `get_projectile_data()`.

You may reuse the constants and `animate(x, y)` helper function from the Lecture 9 problem.

Task

Part 1: Create the parent class

Create a **Projectile** class with these instance attributes:

```
id, size, energy, x_pos, y_pos
```

Set `x_pos` and `y_pos` to 0 by default.

Add a method named `get_projectile_data()` that returns a string with the common projectile data, including its current position.

For example:

```
"id B101, size 30, energy 25, position (0, 0)"
```

Part 2: Create projectile child classes

Create these child classes:

```
class Bomb(Projectile):  
    ...  
  
class Missile(Projectile):  
    ...
```

`Bomb` must add an `explosion_method` attribute and an `explode()` method.

`Missile` must add `fuel` and `guidance_system` attributes and a `guide()` method.

Use `super().__init__(...)` in each child initializer to create the inherited attributes.

Part 3: Use method overriding

Override `get_projectile_data()` in both child classes. Each version must:

1. Call the parent version using `super().get_projectile_data()`.
2. Add the child-specific attributes to the returned string.

For example, a bomb should include its explosion method and a missile should include its fuel and guidance system.

Part 4: Create the `Cannon` class

Create a `Cannon` class with attributes for an identifier and length. Its initializer must receive an existing projectile and store it in an attribute named `self.projectile`.

```
class Cannon:  
    def __init__(self, cannon_id, length, projectile):  
        self.id = cannon_id  
        self.length = length  
        self.projectile = projectile
```

This is aggregation because the projectile is created outside the cannon and passed to it.

Add a `fire(velocity, theta)` method. Move the flight loop from Lecture 9 into this method. The method must update:

```
self.projectile.x_pos
self.projectile.y_pos
```

Use `math.cos`, `math.sin`, gravity, and the time step as in Lecture 9. Call `animate()` once per simulation step.

Part 5: Demonstrate polymorphism

Create one `Bomb` and one `Missile`. Print data for both objects using the same method call:

```
print(bomb.get_projectile_data())
print(missile.get_projectile_data())
```

Then load either object into a cannon and fire it:

```
cannon = Cannon("C1", 20, bomb)
cannon.fire(2, math.radians(45))
```

Finally, print the object's dictionary:

```
print(bomb.__dict__)
```

Requirements

- `Bomb` and `Missile` must inherit from `Projectile`.
- Child initializers must call `super().__init__(...)`.
- `Bomb` and `Missile` must override `get_projectile_data()`.
- `Cannon` must store a projectile received from outside the class.
- `Cannon.fire()` must update the stored projectile's position.
- The same `Cannon` class must be able to fire either a `Bomb` or a `Missile`.
- Reuse the `animate(x, y)` helper from Lecture 9; do not turn it into a class method.

Discussion Questions

1. Why is `fire()` more naturally a method of `Cannon` than a method of `Projectile`?
2. Why is the relationship between `Cannon` and a passed-in projectile aggregation rather than composition?
3. What does `super().__init__(...)` do in `Bomb` and `Missile`?
4. How does calling `get_projectile_data()` demonstrate polymorphism?

5. Which attributes appear in `missile.__dict__`, and why are inherited instance attributes included there?

Extension Challenges

1. Add a `load(projectile)` method so a cannon can replace its current projectile.
2. Change `Cannon` to composition: create a default `Projectile` inside `Cannon.__init__`. Explain how this changes ownership.
3. Add a `Rocket` child class with `fuel` and `engine_power` attributes. Decide whether `Missile` should inherit from `Rocket` or directly from `Projectile`, and justify your choice.